

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

SC4053 BLOCKCHAIN PROJECT

AI Audit Report

Members

Name	Matriculation Number
Francisco Jordon	U2422754C
Seet Jia, Viona	U2322043F
Gerald Wee	U2421571G

Executive summary

A sealed bid ntu domain registrar in solidity and also a React app that connects to Metamask was built by us. The flow is as such, a bidder would first submit a hidden commitment of a fixed 0.1 ETH deposit and during the reveal phase they would disclose the name, bid and salt. The highest valid bid then wins and an earlier commitment wins a tie and after finalization, users can then resolve the name to an address and send ETH to the address.
We used the help of AI assistance like ChatGPT and Codex to turn the brief into checks, compare contract and frontend code and also work through bugs. Whenever a change had been suggested, until relevant tests or browser flow had been done, we did not treat a generated fix as completed. Our most serious problems identified were in the wallet experience, a few of them are recovering a bid after a reload, keeping the right wallet and network context, showing the complete payment recipient and displaying auction deadlines correctly. One important issue was bid recovery and we learnt if a user lost their saved bid details after deposit had been paid, they might not be able to reveal their bid after and this is even when the smart contract is working correctly.

In the end our final local suite passed **182 named tests which is 153 Solidity and 29 Mocha**. We also built and linted the frontend, deployed and verified a fresh local registrar, checked recovery and payment in a browser, and verified the recorded Sepolia deployment, so this report explains what each check showed and where our evidence is limited.

1. Initial Specification & System Scope

So our starting goal taken from Option 3 of the project brief, was an Ethereum testnet registrar for .ntu names. A user should be able to search for a name and also bid without revealing the name or price during commit, reveal later, register the winner, resolve the name to an address and also pay that address through the web app. We chose short phases so the entire auction could be demonstrated, then developed docs/SPEC.md with contract methods, phase rules, accounting, backup format and 29 acceptance checks, then the working specification was refined during implementation and AI-assisted review.

Design decision	Implemented rule	Reason it matters
Hidden commitment	The contract receives round ID, digest and a fixed deposit; the name, bid and salt are disclosed only at reveal.	Makes different bid amounts indistinguishable from the deposit alone and keeps the target name out of public commit calldata and events.

Design decision	Implemented rule	Reason it matters
Round schedule	Explicit start, then 120 seconds for commit and 120 seconds for reveal.	Contract block timestamps, not the browser clock, determine inclusion and validity.
Auction outcome	Highest valid reveal wins; earlier on-chain commitment breaks an equal-bid tie.	Produces a deterministic result independent of reveal ordering.
Deposit accounting	Each commitment locks 0.1 ETH. A winning bidder recovers deposit minus its bid, a valid loser recovers the full deposit, and an unrevealed bidder loses 0.001 ETH.	Keeps refunds and treasury proceeds explicit and prevents the treasury from taking unsettled bidder escrow.
Ownership and payment	Finalization registers the winner; the owner may update a separate resolution address; a payment checks that address during the transaction.	Distinguishes property rights from payment destination and blocks a stale reviewed recipient.
Recovery	A versioned local preimage records chain, registrar, wallet, round, exact wei amount, label, digest and salt before wallet approval.	A bidder needs the same preimage to reveal; losing it after paying the deposit could cause a non-reveal penalty.

The contract decides ownership, reservations, deadlines, results and withdrawal credits, then the frontend prepares the secret commitment details and also stores a recovery copy which then would ask MetaMask to approve transactions. Moving on, browser storage helps a user reveal after a reload, but the contract remains the source of truth, then we also allowed different names to share a round and a later round to begin while an older name still awaited finalization.

Our security checks covered many things like invalid names and amounts, copied commitments, exact deadlines, duplicate actions, unauthorized resolution changes, rejected ETH transfers, reentrancy, changing payment recipients and backup integrity. However, expiry, renewal, subdomains, resale and ownership transfer were outside and not in the base scope, to add on, reverse resolution was a bonus feature.

2. Development Methodology & AI Tooling Infrastructure

Regarding Methodology and AI tooling infrastructure, our project used Solidity and Hardhat 3 for the registrar and tests, a compiler-exported ABI for the frontend interface, and React, Vite, ethers v6 and MetaMask for the application. The team responsibilities on our project were split between the contract, the wallet-connected frontend, and deployment/independent validation. Then, we brought all those branches together only after checking that the ABI, chain settings and user flows are on the same page. Browser feedback on backups, payments and the phase timer shaped the later frontend fixes.

For the work recorded here and ai tools we used, we used ChatGPT/Codex to review the specification and code, compare frontend revisions with the contract ABI, suggest focused tests, diagnose failures, prepare targeted corrections and help draft documentation. Our repeatable checks were repository scripts rather than a custom autonomous agent, team members ran the Windows MetaMask flows, reported what actually appeared, and repeated them after fixes. Decisions then, such as retaining a compatible dependency toolchain and separating browser evidence from contract tests were made after reviewing the results.

Our usual loop was to state the expected behaviour, inspect the relevant source and ABI, reproduce the problem, make a small change, and then rerun the appropriate test. For wallet issues, we also checked the live page, a simulated EVM test can check a contract call, but it cannot prove that MetaMask showed the right approval screen or that a downloaded backup survived a refresh.

Layer	Tool or artifact	How we used it
Contract and integration	Hardhat 3, Solidity tests, Mocha, TypeScript, forge-std	Check auction invariants, time boundaries, money flows, deployment evidence and ABI compatibility.
Frontend and wallet	React, Vite, ethers v6, MetaMask	Inspect name search, preimage persistence, connection state, transaction confirmations and payment review.
Reproducible interfaces	Compiler artifact, contracts/abi/DomainRegistrar.json, docs/CONTRACT_INTERFACE.md	Check method signatures, returned record fields and event fields rather than relying on a stale handwritten interface.
Deployment and evidence	deploy-registrar.ts, verify-deployment.ts, validate.mjs, smoke script, docs/evidence	Bind a real chain/address/treasury and recorded code hashes to commands and results.

Layer	Tool or artifact	How we used it
Historical/current comparison	frontend reproduction harness and FrontendCurrentFlow.ts	Preserve failures from an older snapshot while checking the corrected current handler separately.

The validation command builds the registrar, exports its ABI, typechecks and runs tests, lastly, we also kept a reproduction test for an older frontend version, but pointed the routine regression check at the current handler, if not, an old defect could be mistaken for a defect in the submitted app.

3. Specification Engineering & Prompt Logs

For this section, our specification sets out canonical names, the exact commitment encoding, phase boundaries, reservations, refunds, recovery data and payment review. Two distinctions mattered throughout integration, a commitment ID is not a round ID, and the domain owner may choose a different address to receive payments.

For prompt logs, the table below would summarize the main tasks that AI assistance was used when we did the development, it outlines context which was provided to the AI and how prompts or instructions were refined to tackle specific issues and also how we have done tests and verify the suggested solutions.

Context we gave AI	How we refined the task	What we checked
The registrar contract, generated ABI, specification, deployment script and frontend interface.	Compare canonical method names and record fields; make the chain, treasury, code and ABI independently verifiable; reject a zero treasury.	A 48-entry ABI was exported. Deployment integration tests check evidence mismatches, and local and Sepolia verification returned verified results.
The earlier and updated frontend versions, contract interface and failing handler test output.	Reproduce suspected short-label, round-zero, reservation, owner/recipient and payment defects on the older snapshot; do not carry a historical finding into the current version without retesting.	Five old behaviours were reproduced in a simulated EVM. The newer handler had already fixed several; later current-source checks and browser runs verify the delivered path.
Bid recovery: SPEC backup fields, browser	Save the preimage before asking for approval; bind it to	#1/#2 IDs and timestamps were restored; #7 retained exact wei;

Context we gave AI	How we refined the task	What we checked
import failure, exact 30000000000000001 wei case, chain event and transaction receipt.	chain/registrar/wallet/round; preserve existing valid records; reconcile a pending hash and confirm the correct on-chain commitment.	rejected, pending and settled states were exercised; the final source test injects a failed receipt and RPC interruption.
Payment/network: MetaMask on Sepolia against local chain 31337, checked sendToDomain method, changed-resolution scenario.	Show a useful mismatch error, require a full address/amount/network preview, and reject a recipient changed after preview without automatically retrying.	Wrong-chain actions were blocked; a simulated-EVM stale-recipient test passed; a later local browser race rejected the old preview and displayed a fresh destination.
Phase and compiler follow-up: browser countdown logs, RPC block timestamps, Sepolia creation hash and CRLF/LF difference.	Treat the latest chain block as authoritative for phase actions; distinguish an estimate from a mined deadline; replay the recorded compiler input when checkout line endings alter Solidity metadata.	Browser changed from an incorrect late Reveal display to Awaiting chain block; a reveal mined at revealEnd minus one succeeded. Offline replay matched the recorded Sepolia creation hash.

Two browser tests were carried out to check on how the frontend would handle different situations, when the backup downloads and the webpage confirmation appears, we click Cancel and we do not approve a MetaMask transaction. After that, we then refreshed to check that the bid still said Prepared, not submitted and that earlier records remained. For the network check, we switched MetaMask to Sepolia while the site's local configuration remained on chain 31337, and checked for a visible warning and blocked actions. Also source revision and failure output mattered as much as the words of the request, we checked each suggested cause or fix against current code and actual behaviour.

4. AI-Generated Architecture

The initial system layout we reviewed was one registrar contract and a wallet-connected web app, without an application backend. Then the UI connected through MetaMask to a configured chain and contract address, the registrar kept round timing, commitments, per-name auctions, ownership and resolution, locked deposits and withdrawal credits. AI-assisted review was very useful and what we identified with it was that the first frontend path did not fully account for a rejected approval, a pending transaction, an imported backup or a changed payment recipient, however we addressed these issues. The final architecture was updated on those recovery and confirmation states without replacing the contract's auction model.

Boundary	Data flowing across it	Principal check
Browser to wallet	Proposed transaction and ETH value; locally generated preimage remains in browser backup.	User approval, selected account and selected chain.
Wallet to registrar	A commit digest and fixed deposit, then a reveal with label, amount and salt.	Contract validates round phase, bidder, amount, digest and reservation.
Registrar to wallet/receiver	Settlement creates credit; withdraw transfers caller credit; name payment forwards only that call's value.	Credit is cleared before transfer; payment checks current resolution against reviewed recipient.
Source checkout to deployed chain	ABI, bytecode and recorded compiler input.	Deployment verifier checks network, transaction, runtime code, treasury and ABI.

For a canonical label L, integer bid B and fresh random 32-byte salt S, the browser computes:

H = keccak256(abi.encode(chainId, registrarAddress, roundId, bidderAddress, L, B, S)).

These seven fields and standard ABI encoding bind the hidden bid to a wallet, round, chain and registrar. This makes it so that someone cannot copy the digest and reveal it from a different wallet, the commit transaction exposes the round, digest and fixed deposit, but not the label, amount or salt. The wallet's participation and timing are still visible, and the reveal makes the name and bid public later.

The flow is start round, commit, reveal, finalize each name, settle each bid, and withdraw any credit. In our architecture, a valid bidder settles after finalization and an unrevealed bidder can settle after the round ends, then from the 0.1 ETH deposit D, a winner receives D minus its bid B, a valid loser receives all of D, and a non-revealing bidder receives D minus the 0.001 ETH penalty. The treasury receives the winning price and any penalty through the contract's accounting.

Lastly, finalization makes the winner the owner and initially resolves the name to that wallet. Then after, the owner can later choose a different nonzero receiving address. For a payment, the app shows the resolved address and sends it as the expected recipient to sendToDomain. The contract also checks the current record again before forwarding ETH, so a changed recipient cannot silently replace the one the sender reviewed.

5. Problems Discovered & Vulnerability Analysis

Our most important findings were in the frontend's connection to the contract and wallet, we did not reproduce a new Solidity logic vulnerability and the contract tests cover authorization, reentrancy, accounting and deadlines. However, that does not prove the contract is confirmed to be free of bugs, so for each issue, we explain the behaviour, why the code or workflow appears to have missed it, what we changed and how we checked the fix.

FE-07: Reveal backup and pending receipt recovery

What went wrong and why it mattered. In an early browser check, importing a settled bid lost its on-chain commitment ID and original time. The older frontend also saved a bid only after the commit call finished. If someone paid the fixed deposit and then closed the page before the preimage was saved, they could lose the name, amount or salt needed to reveal and incur the non-reveal penalty. During a separate check, a pending record survived a refresh but remained labelled Submitted after MetaMask reported a failure.

Why we think it happened. The original flow concentrated on a successful transaction and It did not properly treat preparation, wallet approval, broadcast, receipt and chain confirmation as separate steps, this means that things like bid details were not always saved before interruptions occur and the user could lose information that is needed for the reveal phase.

What we changed and checked. The app now saves and offers a backup before asking for webpage and MetaMask approval. Records are separated by chain, registrar, wallet and round. Import checks fields, the exact wei amount and the commitment digest; receipt and chain checks restore the on-chain ID without discarding the secret. We restored earlier #1/#2 IDs and times, preserved the exact 30000000000000001 wei amount for #7, and rejected modified or wrong-context backups without replacing valid records. Canceling confirmation left a Prepared, not submitted record after refresh. On the final combined frontend, a pending bid survived reload, became confirmed ID #1 after mining, and retained its original time; a backup later restored its settled and revealed state. The current-source EVM test separately checked a failed receipt and an RPC interruption after broadcast that retained the transaction hash.

FE-06: Payment confirmation and a changing recipient

What went wrong and why it mattered. An older frontend path sent ETH directly to an address read earlier from the name. If the owner changed that address in between, the sender could pay a destination they had not just reviewed. The later contract method could reject a changed recipient, but the interface still lacked a clear screen showing the complete recipient, amount and network before approval.

Why we think it happened. Even though the contract has checks for the payment recipient, our frontend didn’t display clearly on the entire receiving address and payment details before user confirms the transaction.

What we changed and checked. The app now shows the full address, amount and chain, then calls sendToDomain(label, expectedRecipient). In the EVM test, changing resolution

after preview rejected the stale payment; a fresh preview succeeded. We repeated that race in the browser by changing the resolution in another tab. The old review was rejected, the next review reported a successful 0.001 ETH payment.

FE-09 and FE-10: Network context and visible input validation

What went wrong. When MetaMask was on Sepolia but the site pointed at the local registrar, a read returned an opaque BAD_DATA error. At another point the navbar could call a local connection Sepolia. The old name field also silently stripped an underscore, making the submitted label differ from the text a user typed.

Why we think it happened and how we fixed it. The frontend was not properly checked on whether MetaMask was actually connected to the same network as the configured smart contract and also automatically modified domain names that were invalid instead of showing users what exactly was wrong with their input.

We added explicit chain and deployed-code checks, made the navbar show the active network, and kept invalid text visible so Search could explain the problem. With wallet chain 11155111 and registrar chain 31337, the browser showed the mismatch and blocked search and write actions. Typing bad_name remained visible and produced a syntax message. Contract-side label validation remains an independent check.

FE-11: Countdown disagreed with blockchain phase

What went wrong. We saw the page claim 7m54s remained in a reveal period that lasts only 120 seconds. It offered Reveal when the contract would reject it. Our first clock correction uncovered a second case: the page reached zero while the latest mined block was still five seconds before the on-chain deadline.

Why we think it happened and how we fixed it. The browser's clock and a local test chain's block timestamp can diverge, so we changed the countdown to use chain time as its anchor and to show Awaiting chain block when the estimate has expired but no block has confirmed the next phase. In the browser it moved to Round ended when a new block crossed the deadline. A reveal included one second before its on-chain deadline succeeded with receipt status 1. So, this showed that the transaction’s block timestamp would determine if a reveal is accepted and not the countdown that was displayed on the webpage.

FE-01 and historical interface defects

The production build initially failed because unused frontend paths contained unresolved imports. Removing them let the TypeScript/Vite build pass. We also reproduced five problems in the captured older handler, a two-character label was accepted, round zero was queried before a round existed, an older reservation was misclassified, a resolution address was shown as the owner, and a payment bypassed the checked contract path. Several had already been corrected in a newer frontend revision. We therefore kept that reproduction as a record of what we found, but tested the delivered handler separately instead of reporting old behaviour as a current bug.

DEPLOY-02: Compiler metadata changed across checkouts

The Sepolia deployment was compiled from source with Windows CRLF line endings. Rebuilding the same source from a Linux LF checkout changed the creation-data hash because the compiler metadata changed. That could make verification appear to fail even though the deployed contract was the same. We recorded the original input format and replayed it with the pinned Solidity 0.8.34 compiler and forge-std remapping. The offline LF-checkout test reproduced the recorded creation hash, including constructor arguments. Windows verification against Sepolia separately returned verified true. We did not capture a public-RPC verifier run from the LF checkout.

DEP-01: Development-tool dependency advisories

Our first contract-tooling audit showed 22 affected package entries. We applied a compatible update and reran validation. The 1 November snapshot then showed 13 high and 5 moderate entries, mostly in transitive development tooling. A later Windows install reported 13 high and 3 moderate in contracts and 10 moderate in frontend dependencies, but these are different audit dates and package trees, not one combined count.

We did not claim the remaining advisories were fixed, the suggested direct fixes included major or incompatible Hardhat plugin and Mocha changes, so a forced upgrade would have required another migration and full validation pass. The report records that limitation and an affected package entry is not itself proof of an exploitable flaw in the deployed registrar.

6. Validation Process & Testing Strategy

We tested using different types of testing to check the smart contract and also the frontend. Contract tests check rules even when someone bypasses the UI, frontend integration tests check that our handlers use the ABI and state correctly. Browser checks show what a person actually sees in MetaMask, including canceled approvals and downloaded files. Deployment verification checks that the recorded address and code exist on the intended chain.

Check	Result	Why it mattered
Contract and integration suite	182 passing named tests: 153 Solidity and 29 Mocha. Contract build, ABI export and TypeScript typecheck passed.	We checked auction rules, accounting, deployment binding and the current frontend handler against a local EVM.
Frontend build and lint	Both passed on the combined code revision.	We confirmed the delivered UI compiles and satisfies its configured lint rules.
Fresh local setup	A separate Windows checkout installed both lockfiles, deployed and verified registrar 0x5FbDB2315678afecb367f032d93F	We checked that the README path works with the intended address, chain and ABI.

Check	Result	Why it mattered
	642f64180aa3 on chain 31337, and served the matching frontend.	
Manual browser runs	Two-wallet auctions, equal-bid tie, independent names, backup import, wrong network, changed payment recipient, settlement and withdrawal.	We checked the actual MetaMask and page flow on Hardhat Local.
Sepolia deployment	Registrar 0x9dbe733cDB4F8BBB6a960177c2c3bB7E005819F9 on chain 11155111; deployment transaction 0xb07b99bb1b8e63c1610ab77b9103f1665045a837eb70cd684ca3157915d55408. Windows verification returned verified true.	We confirmed the recorded testnet deployment separately from the local browser demo.
Dependency review	A compatible update reduced the initial audit count, with dated advisories remaining.	We kept the tooling risk visible instead of treating a passing test suite as a clean dependency audit.

The total test count is only part of the evidence. We inspected local commit transactions #7 and #8: each had two arguments and 68 bytes of calldata, while the events showed bidder, round, ID and digest. Neither exposed the name, bid or salt during commit. We also used 30000000000000001 wei as a bid from a 100000000000000000 wei deposit. The winning refund was exactly 69999999999999999 wei, which would catch an accidental conversion through imprecise JavaScript numbers.

In the browser, a 0.06 ETH bid beat 0.04 ETH in round #3. In round #4, two 0.05 ETH bids were revealed in reverse order and the earlier commitment won. Another wallet could finalize the winner's name. We also saw later rounds proceed while an older name was unfinished and checked that earlier credits remained claimable. For actions the UI blocks, such as a second settlement or a nonowner address change, contract tests supplied the direct rejection check. The browser separately showed the disabled or removed control.

The final recovery run was especially useful. Canceling the webpage confirmation left a prepared record after reload. With local mining paused, a 0.02 ETH bid stayed Submitted, checking chain across a second reload. Transaction 0x13a5cd25a6b97c51c60c06e108a3552b3858eae58a9765d5beb521f87c76e989 then mined before the commit deadline with receipt status 1. The page recovered ID #1 and kept its original 5:22:56 PM time. We revealed the bid, finalized the name and settled it for a 0.08 ETH credit. Withdrawal

0x18b21ffe290156a19ab1139a5588717d60ea65d121eac83afabbf4254da90bc5 had receipt status 1 and left zero credit. After removing only that local record, importing its backup restored the settled status, reveal status, ID and time without losing older records. The failed-receipt and post-broadcast RPC interruption cases were exercised in the current-source EVM test rather than that browser run.

Earlier validation logs show 176–181 tests as the suite grew. The final combined code has **182** passing named tests, including five baseline Counter examples. Its command output is in the project's dated validation logs. The browser work above used Hardhat Local, it does not stand in for a complete Sepolia frontend run.

7. Improvements & Refinements

We replaced a handwritten interface assumption with an ABI exported from the compiler, deployment records now connect the chain, address, treasury, source, ABI, runtime code and transaction. The fresh-checkout exercise and CRLF compiler replay made it possible to verify that connection on another machine instead of assuming that a local build matched the Sepolia deployment.

The bid flow now treats preparation, submission, confirmation and failure as different states. A user can save the preimage before approving; a broadcast record keeps its hash while waiting for a receipt; a confirmed record gains an on-chain ID. Import checks the exact amount and wallet/chain context before merging it into existing history. This was more work than a single optimistic commit successful message, but it addressed the failure we actually saw after reload.

For payments, we added a clear review step and bound the displayed recipient to the contract call. For deadlines, we let the blockchain phase decide what the app can offer and treat the visible timer as an estimate. We also changed the routine frontend regression check to run on the current handler while retaining the old reproduction for historical comparison. None of these integration repairs required rewriting the original Solidity auction rules.

8. Lessons Learned

Where AI helped. ChatGPT/Codex was effective when we gave it a small, checkable task and the relevant contract, ABI and failure output. So, it helped us turn the broad registrar brief into concrete cases, compare old and current frontend behaviour, propose an exact-wei test and reason about backup and deployment verification. This saved time in finding places to inspect and the useful output was usually a hypothesis or a narrow patch, followed by a test we could run ourselves.

Where it fell short. Code review alone did not catch the full wallet experience and problems were not very obvious when reviewing code alone, we discovered lost backup metadata, a canceled approval that still needed a saved preimage, a wrong-network message and a stale payment review by using the browser. AI-assisted analysis also had to be kept tied to a source revision, a defect reproduced on an archived handler was not necessarily still present in the

current app. Lastly, the clock issue was another reminder that an apparently reasonable frontend calculation can be wrong when the local EVM and computer clocks diverge.

How we kept control of the result. We checked proposed fixes against the compiled ABI, reran targeted and full tests, examined transaction receipts where available, and then repeated the wallet flow, the most important example was bid recovery, a passing hash test did not tell us whether a user could reveal after closing the page. Only a prepared backup, reload, pending receipt and restored record demonstrated that path. We also kept testnet verification separate from the local browser evidence instead of treating either as proof of the other.

What we would change next time. What we would change the next time is that we would design recovery before the first paid wallet action, test rejected approvals and reloads early, and require a visible full-recipient confirmation before payments. We would also record the chain ID, contract address and compiler input alongside every deployment check.